



PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería – Departamento de Ingeniería de Sistemas

**PLAN DE ADMINISTRACIÓN DEL PROYECTO DE SOFTWARE
(SPMP)**

**Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones
en Localidades de Bogotá**

Estándar base: IEEE Std 1058-1998 / ISO/IEC 12207:2008

Samuel Esteban Campos
Erick Salazar Suárez
Santiago Chitiva Contreras
Felipe Andrés Garrido Flores

Director: Ing. Andrés Oswaldo Calderón Romero, PhD.

Bogotá D.C., 11 de mayo de 2026

1. Historial de Cambios

Fecha	Descripción del cambio	Autor(es)
30/04/2026	Creación inicial del documento. Definición de estructura y alcance preliminar del proyecto.	Santiago Chitiva, Samuel Campos
01/05/2026	Actualización del propósito del proyecto tras reunión con el director.	Felipe Garrido, Erick Salazar
04/05/2026	Incorporación del modelo de ciclo de vida híbrido (CRISP-DM + Scrum). Justificación de alternativas descartadas.	Samuel Campos, Santiago Chitiva
05/05/2026	Incorporación de comentarios de la Feria de Pósters. Ajuste de estimaciones Fase 3. Confirmación del alcance con director.	Felipe Garrido, Erick Salazar
06/05/2026	Revisión y aprobación por el director. Ajustes al plan de riesgos (R1 y R3 reclasificados como críticos).	Santiago Chitiva, Samuel Campos
09/05/2026	Actualización de la calendarización a 10 semanas con incorporación de ruta crítica y prerequisites	Todos
10/05/2026	Versión final para entrega formal. Revisión de consistencia interna entre secciones.	Erick Salazar, Felipe Garrido
31/05/2026	Reincorporación formal del módulo web al alcance del proyecto por indicación del director y la profesora. Actualización de secciones 6.1, 6.2, 6.3, 6.5, WBS (F3.5), presupuesto y criterios de aceptación. Eliminación de restricción que excluyó la interfaz web en la versión anterior.	Todos
02/06/2026	Correcciones derivadas de las sugerencias del Profesor: adición de paginación, renumeración de figuras, ajuste de figuras a márgenes, inclusión de captions en figuras y tablas, y verificación de referencias citadas en el texto.	Todos

2. Prefacio

El presente documento constituye el Plan de Administración del Proyecto de Software (SPMP) para el proyecto de grado “Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá”, desarrollado por cuatro estudiantes de Ingeniería de Sistemas de la Pontificia Universidad Javeriana.

El proyecto tiene como propósito diseñar e implementar dos módulos. Un módulo de procesamiento de datos que automatice la transformación de información proveniente de la Encuesta de Movilidad de Bogotá (2023) y de aforos vehiculares en hora de máxima demanda, en archivos de entrada válidos para el simulador de tráfico SUMO (Simulation of Urban MObility) [4] y un módulo web que permite visualizar los resultados producidos por SUMO y parametrizar nuevas configuraciones de simulaciones desde una interfaz gráfica accesible, sin necesidad de interactuar directamente con archivos de configuración técnicos.

Este documento está dirigido principalmente al equipo de desarrollo, al director del proyecto y a los jurados evaluadores del trabajo de grado. El SPMP se basa en el estándar IEEE Std 1058-1998 [1], complementado con ISO/IEC 12207:2008 [2]. Los procesos de soporte se describen con diagramas BPMN 2.0 [3], cuya ubicación se señala en cada sección correspondiente.

3. Tabla de Contenidos

1. Historial de Cambios
2. Prefacio
3. Tabla de Contenidos
4. Lista de Figuras
5. Lista de Tablas
6. Vista General del Proyecto
 - 6.1 Visión del Producto
 - 6.2 Propósito, Alcance y Objetivos
 - 6.3 Supuestos y Restricciones
 - 6.4 Entregables
 - 6.5 Resumen de Calendarización y Presupuesto
 - 6.6 Evolución del Plan
7. Glosario
8. Contexto del Proyecto
 - 8.1 Modelo de Ciclo de Vida
 - 8.2 Lenguajes y Herramientas
 - 8.3 Plan de Aceptación del Producto
 - 8.4 Organización del Proyecto y Comunicación
9. Administración del Proyecto
 - 9.1 Métodos y Herramientas de Estimación
 - 9.2 Inicio del Proyecto
 - 9.3 Planes de Trabajo del Proyecto
10. Monitoreo y Control del Proyecto
11. Entrega del Producto
12. Procesos de Soporte
13. Anexos
14. Referencias

4. Lista de Figuras

Figura 1. Diagrama alto nivel modelo ciclo de vida

Figura 2. Diagrama BPMN – Modelo de ciclo de vida híbrido CRISP-DM / Scrum.

Figura 3. Diagrama BPMN – Proceso de administración de requerimientos.

Figura 4. Diagrama BPMN – Proceso de identificación y seguimiento de riesgos.

Figura 5. Diagrama BPMN – Proceso de control de configuración.

Figura 6. Diagrama BPMN – Proceso de control de calidad.

Figura 7. Organigrama del equipo de desarrollo.

Figura 8. Diagrama WBS

5. Lista de Tablas

Tabla 1. Historial de cambios del documento.

Tabla 2. Entregables del proyecto.

Tabla 3. Calendarización con camino crítico (WBS + prerequisites).

Tabla 4. Asignación de recursos por fase.

Tabla 5. Resumen de presupuesto (horas-persona).

Tabla 6. Herramientas y lenguajes del proyecto.

Tabla 7. Criterios de aceptación de entregables.

Tabla 8. Entidades externas e interfaces.

Tabla 9. Roles del equipo y responsabilidades.

Tabla 10. Entrenamiento del personal.

Tabla 11. Infraestructura y configuración.

Tabla 12. Registro de riesgos identificados.

Tabla 13. Plan de mitigación de riesgos prioritarios.

Tabla 14. Ítems de configuración.

Tabla 15. Ciclo de vida de artefactos.

Tabla 16. Métricas del proyecto.

Tabla 17. Procesos de control de calidad.

6. Vista General del Proyecto

6.1 Visión del Producto

Bogotá enfrenta uno de los problemas de congestión vehicular más severos del mundo: según el TomTom Traffic Index 2025, ocupa el séptimo lugar global con un índice del 69.9% y velocidades promedio en horas pico que apenas superan los 15 km/h. Pese a la existencia de herramientas de simulación de tráfico de clase mundial como SUMO [4], su uso por parte de planificadores y analistas de movilidad está limitado por la alta complejidad técnica que implica transformar datos reales de movilidad en los formatos de entrada que el simulador requiere.

La visión del producto es reducir esa barrera con un módulo de procesamiento de datos que, a partir de la Encuesta de Movilidad de Bogotá 2023 [6] y datos de aforos vehiculares, genere automáticamente los archivos necesarios para ejecutar simulaciones en SUMO. El sistema permite a investigadores y analistas explorar escenarios hipotéticos de movilidad sin necesidad de intervención manual en la preparación de datos.

El alcance de la versión 1 del proyecto está delimitado a la localidad de Usaquén como caso piloto, con un diseño orientado a escalar a todas las localidades de Bogotá en versiones futuras. El sistema comprende dos módulos: un módulo de procesamiento de datos (pipeline ETL) y un módulo web para la visualización interactiva de resultados y la parametrización de nuevas simulaciones, ambos incluidos en el alcance actual por instrucción del director del proyecto.

6.2 Propósito, Alcance y Objetivos

Propósito

El proyecto busca eliminar la brecha entre los datos disponibles de movilidad urbana en Bogotá y las herramientas de simulación de tráfico. La transformación de datos de la Encuesta de Movilidad en insumos para SUMO requiere actualmente procesos manuales que toman horas de trabajo especializado. El módulo desarrollado automatiza este proceso de extremo a extremo.

Alcance – Incluye

- Análisis y comparación de fuentes de datos de movilidad disponibles (Encuesta de Movilidad 2023, aforos vehiculares HMD, OpenStreetMap).
- Desarrollo del pipeline ETL que transforma los datos en archivos de entrada de SUMO: red vial (.net.xml), zonas TAZ (.taz.xml), matrices O/D (.od), archivos de rutas (.rou.xml) y tipos de vehículo (.vtype.xml).
- Validación del pipeline en la localidad de Usaquén, verificando coherencia con aforos reales mediante el estadístico GEH.
- Análisis de al menos dos escenarios hipotéticos de movilidad (infraestructura vial y control de tráfico).

- Desarrollo de un módulo web que permita visualizar de forma interactiva los resultados producidos por SUMO (métricas de congestión, tiempos de viaje) y parametrizar nuevas simulaciones desde una interfaz gráfica, sin necesidad de editar archivos manualmente.

Alcance – Excluye

- Integración en tiempo real con APIs de tráfico comerciales.
- Despliegue en sistemas de producción institucionales.
- Calibración del modelo para localidades distintas a Usaquén en esta versión.

Objetivo General

Diseñar e implementar un módulo de procesamiento de datos que automatice la transformación de fuentes de datos de movilidad disponibles para Bogotá en archivos de entrada válidos para el simulador SUMO, eliminando la intervención manual en la generación de redes viales, matrices origen-destino y archivos de rutas.

Objetivos Específicos

- Analizar y comparar las fuentes de datos de movilidad disponibles para Bogotá para determinar cuáles permiten construir matrices O/D suficientemente representativas [5].
- Desarrollar el módulo de procesamiento que automatice la transformación en archivos válidos para SUMO, con manejo de datos faltantes y resultados reproducibles.
- Validar el módulo generando insumos de simulación para la localidad de Usaquén, verificando coherencia con aforos reales y ejecutando el escenario en SUMO.
- Desarrollar un módulo web que permita visualizar de forma interactiva los resultados generados por SUMO y parametrizar nuevas simulaciones desde una interfaz gráfica, reduciendo la barrera técnica de acceso al sistema para usuarios del área de movilidad.

6.3 Supuestos y Restricciones

Supuestos

- La Encuesta de Movilidad de Bogotá 2023 y la base de datos de aforos vehiculares HMD son accesibles y de calidad suficiente para construir matrices O/D representativas.
- La red vial de Bogotá en OpenStreetMap tiene suficiente cobertura y atributos para generar una red simulable en SUMO para la localidad de Usaquén.
- Los cuatro integrantes del equipo están disponibles durante las 16 semanas del proyecto con una dedicación aproximada de 10 horas semanales por persona.
- El equipo de cómputo disponible tiene al menos 8 GB de RAM y un procesador de 4 núcleos para ejecutar SUMO a escala de localidad.

- SUMO en su versión estable 1.19+ soporta todos los modos de transporte a modelar.

Restricciones

- El proyecto está planeado para completarse en 10 semanas de trabajo efectivo. Aunque el semestre cuenta con 16 semanas para la entrega final, la calendarización se diseñó dejando semanas adicionales de contingencia y ajuste, con el fin de disponer de tiempo suficiente para corregir errores, realizar pruebas, atender imprevistos y efectuar mejoras antes de la entrega final.
- Todas las herramientas deben ser de código abierto o de uso académico gratuito (sin licencias comerciales).
- El alcance geográfico está limitado a la localidad de Usaquén para la validación.
- El código fuente debe seguir la guía de estilo PEP 8 y estar versionado en GitHub.
- El módulo web debe ser accesible desde navegadores modernos sin instalación adicional; no se contempla despliegue en producción institucional en esta versión.

6.4 Entregables

Entregable	Estándar	Descripción	Audiencia	Semana
SPMP corregido	IEEE 1058-1998	Plan de administración del proyecto	Equipo / Director / Jurados	3
SRS corregido	IEEE 830-1998	Especificación de requisitos del software (IEEE 830 [9])	Equipo / Director / Jurados	4
SDD	IEEE 1016-2009	Documento de diseño del sistema (IEEE 1016 [10])	Equipo / Director	6
Plan de Pruebas	ISO/IEC/IEEE 29119	Plan detallado de pruebas (ISO/IEC/IEEE 29119 [11])	Equipo / Director	7
Código Fuente	PEP 8 / API TraCI	Pipeline ETL en Python + scripts SUMO	Repositorio GitHub	9
Informe Final	IEEE 1063-2001	Documento final con resultados	Jurados / Universidad	10-16

6.5 Resumen de Calendarización y Presupuesto

Hitos del Proyecto

Hito	Descripción	Semana
------	-------------	--------

H1	Entrega SPMP y SRS corregidos. Arranque formal del proyecto	1-2
H2	Pipeline ETL funcional. Matrices O/D y red vial generadas	2-6
H3	Módulo web funcional: visualización de resultados y parametrización de simulaciones	6-8
H4	Entrega SDD y Plan de Pruebas	8-10
H5	Escenario base validado en SUMO (Sprint 1)	10-12
H6	Escenarios hipotéticos analizados (Sprint 3-4)	12-14
H7	Entrega informe final	14-16

Resumen de Presupuesto

Ítem	Descripción	Cantidad	Costo equivalente
Horas-persona totales	4 personas × 10 semanas × 14 h/semana	560 h	—
Herramientas de software	SUMO, Python, Angular, GitHub, GitHub Actions (todos gratuitos)	—	\$0 COP
Datos de entrada	Encuesta Movilidad 2023, aforos, OSM (todos públicos)	—	\$0 COP
Infraestructura de cómputo	Equipos personales del equipo + Google Colab	—	\$0 COP
Total monetario del proyecto			\$0 COP

El proyecto no maneja presupuesto monetario. Todas las herramientas son de código abierto y los datos de entrada son de acceso público. El costo total equivalente se expresa en horas-persona: 4 integrantes × 10 semanas × 14 h/semana ≈ 560 horas-persona.

6.6 Evolución del Plan

El presente documento es un artefacto vivo que se actualizará en cada entrega formal del proyecto. Los cambios al plan se gestionan a través del proceso de control de configuración descrito en la sección 12.3. Cualquier modificación sustancial al alcance, calendario o recursos debe ser aprobada por el director y registrada en el historial de cambios (sección 1).

7. Glosario

Término / Acrónimo	Definición
SUMO	Simulation of Urban MObility. Simulador de tráfico microscópico de código abierto.
Fast API	Framework de desarrollo web
ETL	Extract, Transform, Load. Pipeline de procesamiento de datos por etapas.
O/D	Matriz Origen-Destino. Estructura que cuantifica viajes entre pares de zonas geográficas.
TAZ	Traffic Analysis Zone. Unidad espacial básica del modelo de demanda de transporte.
UPZ	Unidad de Planeamiento Zonal. División administrativa de Bogotá usada como TAZ.
HMD	Hora de Máxima Demanda. Período de mayor volumen vehicular (AM o PM).
GEH	Estadístico de validación de modelos de tráfico. Valor < 5 indica buena calibración.
RMSE	Root Mean Square Error. Métrica de error cuadrático medio usada en validación.
CRISP-DM	Cross-Industry Standard Process for Data Mining. Metodología iterativa de ciencia de datos.
Scrum	Marco ágil de desarrollo de software basado en sprints iterativos cortos.
TraCI	Traffic Control Interface. API de SUMO para control en tiempo real de simulaciones.
netconvert	Herramienta de SUMO para convertir redes viales de OSM al formato .net.xml.
duarouter	Herramienta de SUMO para asignar rutas a los viajes de las matrices O/D.
OSM	OpenStreetMap. Mapa vial colaborativo de código abierto.
PEP 8	Guía de estilo para código Python de la Python Software Foundation.
WBS	Work Breakdown Structure. Estructura de descomposición de tareas del proyecto.
SPMP	Software Project Management Plan. Plan de Administración del Proyecto de Software.
SRS	Software Requirements Specification. Especificación de Requisitos del Software.
SDD	Software Design Document. Documento de Diseño del Software.

8. Contexto del Proyecto

8.1 Modelo de Ciclo de Vida

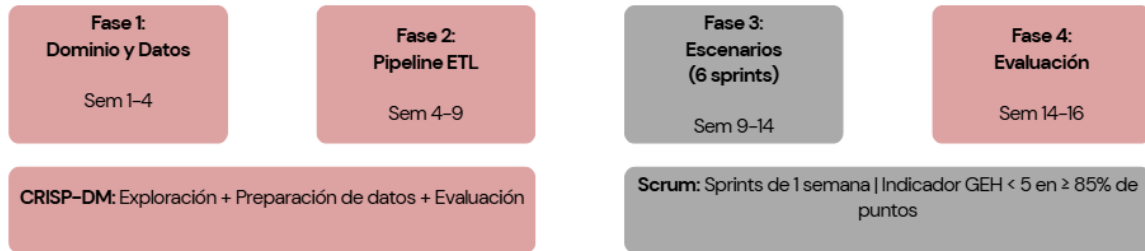


Figura 1. Diagrama alto nivel del modelo de ciclo de vida.

Descripción del Modelo

El proyecto adopta un modelo de ciclo de vida híbrido que combina CRISP-DM [7] para las fases intensivas en datos (Fases 1 y 2) con Scrum [8] para la construcción iterativa del escenario de simulación (Fase 3). La Fase 4 de evaluación retoma la estructura de CRISP-DM (véase la Figura 1).

Las Fases 1 y 2 siguen CRISP-DM porque el trabajo en esa etapa es de naturaleza exploratoria y analítica: comprender las fuentes de datos, evaluar su calidad, identificar brechas y construir el pipeline ETL. A partir de la Fase 3, el proyecto migra a Scrum con sprints semanales, dado que el trabajo pasa a ser incremental y orientado a componentes verificables de software. Además, se aplicará el tablero visual de la metodología Kanban para gestionar las tareas

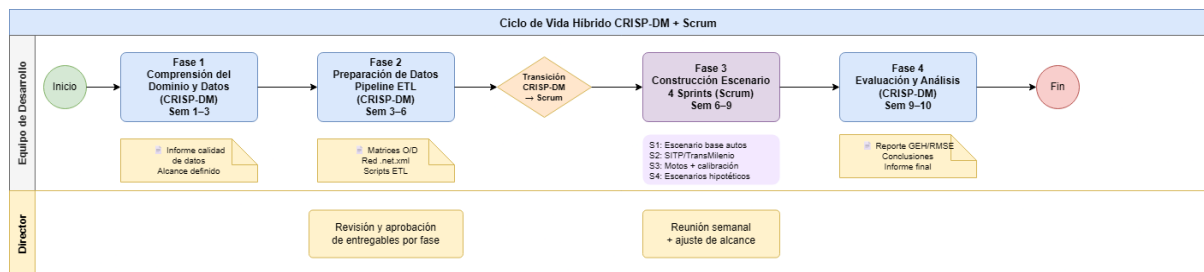


Figura 1. Diagrama BPMN – Modelo de ciclo de vida híbrido CRISP-DM / Scrum

Figura 2. Diagrama BPMN – Modelo de ciclo de vida híbrido CRISP-DM / Scrum.

[DIAGRAMA BPMN – Figura 2 – Ciclo de vida híbrido CRISP-DM + Scrum] Muestra las 4 fases del proyecto, la transición entre metodologías, los roles involucrados en cada fase y los artefactos que produce cada una (matrices O/D, red .net.xml, código Python, reportes GEH). El diagrama debe ser consistente con el WBS de la sección 9.3.1.

Análisis de Alternativas

Se evaluaron los siguientes modelos de ciclo de vida antes de seleccionar el híbrido:

- Cascada: descartado por su rigidez ante cambios en la calidad de los datos de entrada, que no se conoce hasta la exploración.
- CRISP-DM puro: insuficiente para la Fase 3, que requiere gestión de sprints y entregas incrementales de software.
- RUP: se descartó por su excesiva burocracia que podría frenar la agilidad del análisis.
- **Híbrido CRISP-DM + Scrum+ Kanban:** modelo que combina exploración de datos iterativa con entregas incrementales de software

Prácticas Específicas

- Control de versiones: Git + GitHub con ramas por sprint (main, develop, feature/*).
- Revisiones de código: revisión obligatoria de al menos un compañero antes de hacer merge a cada cambio.
- Integración continua: GitHub Actions para ejecutar pruebas unitarias automáticamente en cada commit.
- Reuniones de equipo: dos reuniones semanales (lunes: planeación/seguimiento; viernes: revisión).
- Reunión con director: una vez a la semana, al cierre de cada sprint.
- Definición de 'done' para simulación: componente validado con GEH < 5 en al menos el 85% de los puntos de aforo.

8.2 Lenguajes y Herramientas

Herramienta	Propósito	Justificación
Python 3.11+	Lenguaje principal del pipeline ETL y scripts de simulación	Ecosistema estándar para análisis de datos. Requerido por la API TraCI de SUMO.
Fast API 0.136	Framework para el desarrollo del módulo web	Es una herramienta flexible y fácil de utilizar y configurar para desarrollar el módulo web.
SUMO 1.19+	Motor de simulación microscópica de tráfico	Herramienta de código abierto estándar. Única alternativa open source con soporte multimodal completo.
pandas / geopandas	Procesamiento de datos tabulares y geoespaciales	Estándar de Python para ETL de datos estructurados y geoespaciales.
OpenStreetMap + osmium	Fuente y procesamiento de la red vial	Única fuente gratuita con cobertura global y nivel de detalle suficiente.
netconvert / duarouter	Conversión OSM a .net.xml y generación de rutas	Herramientas oficiales del ecosistema SUMO; garantizan compatibilidad.

Git + GitHub	Control de versiones y repositorio del proyecto	Estándar de la industria. Permite colaboración distribuida y control de cambios.
GitHub Actions	Integración continua (CI)	Gratuito. Automatiza la ejecución de pruebas en cada commit.
pytest	Pruebas unitarias del pipeline ETL	Estándar de facto para pruebas en Python. Integración nativa con GitHub Actions.
Jupyter Notebooks	Exploración y caracterización de datos (Fase 1)	Permite análisis exploratorio iterativo con visualización integrada.
QGIS	Validación geoespacial de la red vial y TAZ	Herramienta GIS de código abierto; integración con OSM y geopandas.

8.2.1 Análisis de Alternativas y Justificación

8.3 Plan de Aceptación del Producto

Entregable	Criterio de aceptación	Herramienta de verificación
SPMP	Cubre todas las secciones del estándar IEEE 1058. Consistencia interna entre secciones. Aprobado por el director.	Revisión documental del director
SRS	Requerimientos trazables, no ambiguos y verificables. Matriz de trazabilidad incluida.	Revisión formal + checklist IEEE 830
SDD	Arquitectura documentada. Decisiones de diseño justificadas. Trazabilidad con SRS.	Revisión formal del director
Plan de Pruebas	Cobertura del 100% de los requerimientos funcionales del SRS.	Matriz de trazabilidad SRS-pruebas
Código fuente	Pasa el 90% de las pruebas unitarias. Cumple PEP 8. Documentado con docstrings.	pytest + GitHub Actions + pylint
Pipeline validado	GEH < 5 en al menos el 85% de los puntos de aforo. Ejecutable en SUMO sin errores.	Métricas GEH y RMSE vs. aforos reales
Informe final	Incluye todos los elementos del IEEE 1063-2001. Aprobado por jurados.	Revisión de jurados en la exposición
Módulo web	Permite visualizar métricas de SUMO. Permite lanzar una simulación parametrizada exitosamente desde la interfaz	Prueba funcional manual + revisión del director

8.4 Organización del Proyecto y Comunicación

8.4.1 Interfaces Externas

Entidad	Rol en el proyecto	Medio de contacto
Ing. Andrés Calderón Romero (Director)	Revisor, aprobador de entregables y asesor técnico	andrescalderonr@javeriana.edu.co / Teams
Secretaría Distrital de Movilidad	Propietaria de los datos de entrada (encuesta y aforos)	Portal de datos abiertos de Bogotá
Jurados evaluadores	Evaluación final del proyecto de grado	Coordinación académica
OpenStreetMap Community	Fuente de la red vial base del modelo	openstreetmap.org

8.4.2 Roles del Equipo

La estructura organizacional del equipo se presenta en la Figura 7.

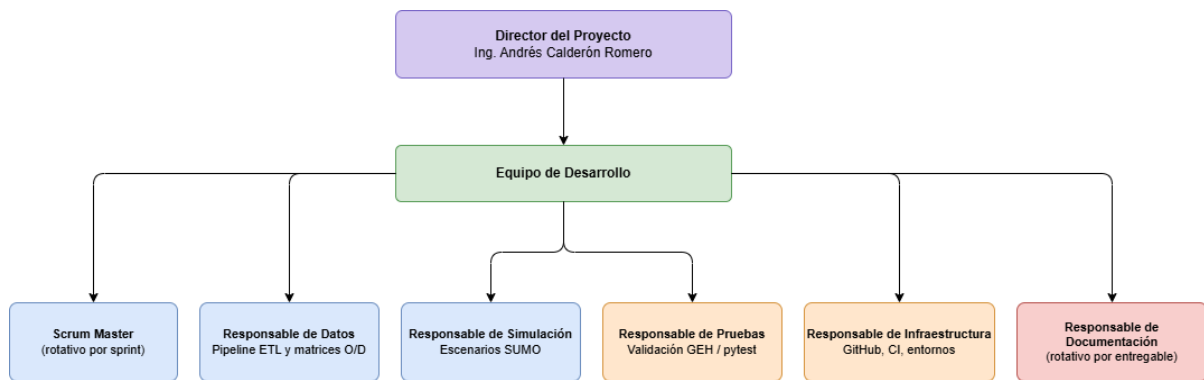


Figura 7. Organigrama del equipo de desarrollo

Rol	Integrante(s)	Responsabilidades principales
Scrum Master (rotativo por sprint)	Rotación entre integrantes	Facilitar ceremonias Scrum y actualizar burndown chart.
Responsable de datos		Liderar Fases 1 y 2 (CRISP-DM): exploración, limpieza, matrices O/D.
Responsable de simulación		Liderar Fase 3 (Scrum): construcción y calibración del escenario en SUMO.
Responsable de documentación	Rotación por entregable	Coordinar redacción y revisión de cada entregable formal.
Responsable de pruebas		Definir y ejecutar pruebas unitarias del pipeline y validación del modelo.
Responsable de infraestructura		Configurar y mantener GitHub, Actions, entornos Python y SUMO.

Scrum Master

Integrantes responsables: Rotativo por sprint entre todos los integrantes.

El rol de Scrum Master se implementa de manera rotativa para garantizar que todos los integrantes participen activamente en la gestión del proyecto y comprendan el estado global del desarrollo. Cada sprint tiene un Scrum Master diferente, responsable de coordinar las actividades del equipo durante ese período.

Responsabilidades principales:

- Planificar y moderar las reuniones semanales de seguimiento.
- Coordinar la reunión de planeación al inicio de cada sprint.
- Actualizar el backlog del proyecto en GitHub Projects.
- Mantener actualizado el burndown chart y las métricas de avance.
- Verificar que las tareas asignadas tengan responsables y fechas definidas.
- Identificar bloqueos técnicos u organizacionales.
- Escalar problemas críticos al director cuando sea necesario.
- Facilitar la comunicación entre los integrantes.
- Coordinar retrospectivas al cierre de cada sprint.
- Garantizar que las actividades priorizadas estén alineadas con los objetivos del sprint.

Responsabilidades de comunicación:

El Scrum Master es responsable de consolidar el estado semanal del proyecto y comunicarlo durante las reuniones con el director. También debe asegurar que todas las decisiones importantes queden registradas en GitHub Issues o en la documentación formal correspondiente.

Entregables asociados:

- Backlog actualizado.
- Burndown charts.
- Registro de reuniones.
- Registro de retrospectivas.
- Seguimiento de issues y tareas.

Responsable de Datos

El Responsable de Datos lidera las actividades relacionadas con adquisición, limpieza, análisis, transformación y validación de los datos de entrada utilizados por el sistema.

Este rol tiene especial relevancia durante las Fases 1 y 2 del proyecto, correspondientes a comprensión del dominio y preparación de datos bajo CRISP-DM.

Responsabilidades principales:

- Analizar la estructura de la Encuesta de Movilidad 2023.
- Comprender y documentar la estructura de los aforos vehiculares HMD.
- Validar calidad, consistencia y cobertura de los datos.
- Diseñar el pipeline ETL de procesamiento.
- Implementar scripts de limpieza y transformación.
- Aplicar factores de expansión muestral.
- Construir matrices origen-destino.
- Verificar consistencia estadística de los resultados.
- Gestionar los datasets del proyecto.
- Documentar supuestos y limitaciones de los datos.
- Coordinar validaciones cruzadas entre aforos y matrices O/D.

Responsabilidades técnicas:

El Responsable de Datos también debe garantizar que los formatos generados sean compatibles con las herramientas del ecosistema SUMO, particularmente con duarouter y netconvert.

Asimismo, debe asegurar que los scripts sean reproducibles y correctamente documentados, permitiendo que cualquier integrante pueda ejecutar nuevamente el pipeline con los mismos resultados.

Herramientas asociadas:

- Python
- pandas
- geopandas
- Jupyter Notebooks
- OpenStreetMap
- GitHub
- Entregables asociados
- Scripts ETL.
- Matrices O/D.
- Reportes de calidad de datos.
- Archivos procesados de entrada.
- Validaciones estadísticas.

Responsable de Simulación:

El Responsable de Simulación lidera todas las actividades relacionadas con la construcción, calibración y ejecución de escenarios dentro del simulador SUMO. Este rol tiene especial relevancia durante la Fase 3 del proyecto, correspondiente a la construcción iterativa del escenario de simulación.

Responsabilidades principales:

- Configurar SUMO y las herramientas complementarias.
- Procesar la red vial mediante netconvert.
- Configurar escenarios de simulación.
- Generar archivos .rou.xml y .vtype.xml.
- Integrar modos de transporte dentro del simulador.
- Ajustar parámetros de comportamiento vehicular.
- Ejecutar simulaciones de validación.
- Analizar resultados de congestión y movilidad.
- Implementar escenarios hipotéticos.
- Verificar que las simulaciones sean reproducibles.
- Documentar parámetros de calibración utilizados.

Responsabilidades de validación:

El Responsable de Simulación debe verificar que las simulaciones ejecuten correctamente y que los resultados producidos mantengan coherencia con los patrones observados en los datos reales. Además, debe coordinar la validación mediante métricas GEH y RMSE junto con el Responsable de Pruebas.

Herramientas asociadas:

- SUMO
- netconvert
- duarouter
- TraCI
- Python
- QGIS
- Entregables asociados
- Escenarios SUMO.
- Archivos de rutas.
- Archivos de tipos de vehículo.
- Resultados de simulaciones.
- Reportes de calibración.
- Escenarios hipotéticos.

Responsable de Pruebas y Validación

El Responsable de Pruebas y Validación tiene como propósito garantizar la calidad técnica del sistema y verificar que los resultados producidos por el pipeline sean consistentes y correctos.

Responsabilidades principales:

- Diseñar casos de prueba del pipeline ETL.
- Diseñar pruebas unitarias para scripts críticos.
- Configurar pytest y GitHub Actions.
- Ejecutar pruebas automáticas.
- Analizar resultados de cobertura.
- Validar consistencia de archivos generados.
- Verificar métricas GEH y RMSE.
- Coordinar validación funcional del pipeline.
- Registrar defectos encontrados.
- Realizar seguimiento de errores reportados.
- Verificar corrección de errores.

Responsabilidades de calidad:

Este rol debe asegurar que cada incremento del proyecto cumpla con la definición de “done” establecida por el equipo.

También es responsable de garantizar que los cambios realizados no introduzcan regresiones en funcionalidades previamente validadas.

Herramientas asociadas:

- pytest
- GitHub Actions
- pylint
- Python
- GitHub Issues
- Entregables asociados
- Casos de prueba.
- Reportes de cobertura.
- Registro de defectos.
- Validaciones GEH.
- Resultados de pruebas.

Responsable de Infraestructura

El Responsable de Infraestructura administra los entornos de desarrollo y garantiza que el equipo disponga de las herramientas necesarias para ejecutar el proyecto de manera estable y reproducible.

Responsabilidades principales:

- Configurar el repositorio principal en GitHub.
- Definir estructura de carpetas del proyecto.
- Configurar GitHub Actions.
- Mantener requirements.txt actualizado.
- Configurar entornos virtuales Python.
- Gestionar versiones de SUMO.
- Verificar compatibilidad entre herramientas.
- Gestionar almacenamiento de datasets.
- Administrar permisos y ramas del repositorio.
- Verificar respaldos y versionamiento.

Responsabilidades operativas:

El Responsable de Infraestructura debe apoyar a los demás integrantes cuando existan problemas de instalación, dependencias o conflictos de configuración. También debe garantizar que la infraestructura utilizada permita la ejecución reproducible del pipeline en diferentes equipos.

Herramientas asociadas:

- GitHub
- Git
- GitHub Actions
- Python
- SUMO
- Google Colab
- Entregables asociados
- Repositorio configurado.
- Integración continua funcional.
- Entornos reproducibles.
- Configuración de herramientas.

Responsable de Documentación

Integrantes responsables: Rotativo por entregable.

El Responsable de Documentación coordina la elaboración, revisión y consolidación de todos los documentos formales del proyecto.

Responsabilidades principales:

- Coordinar la redacción de entregables.

- Verificar consistencia entre documentos.
- Mantener formato y estructura de estándares IEEE.
- Consolidar diagramas y tablas.
- Actualizar historial de cambios.
- Gestionar referencias bibliográficas.
- Revisar ortografía y redacción.
- Verificar trazabilidad entre documentos.
- Coordinar revisiones internas.
- Incorporar observaciones del director.

Responsabilidades de calidad documental:

El Responsable de Documentación debe garantizar que todos los documentos mantengan coherencia con el estado actual del proyecto y con las decisiones tomadas durante el desarrollo.

Asimismo, debe asegurar que las versiones entregadas cumplan con los lineamientos establecidos por la Pontificia Universidad Javeriana y los estándares IEEE seleccionados.

Entregables asociados:

- SPMP.
- SRS.
- SDD.
- Plan de pruebas.
- Informe final.
- Diagramas BPMN.
- Tablas y anexos.

Responsabilidad compartida del equipo

Aunque existen roles especializados, el proyecto adopta un enfoque colaborativo donde todos los integrantes participan activamente en decisiones críticas relacionadas con arquitectura, calidad, alcance y validación.

Las siguientes actividades son responsabilidad compartida del equipo:

- Participación en reuniones semanales.
- Revisión de código.
- Revisión de documentación.
- Discusión de decisiones técnicas.
- Identificación de riesgos.
- Resolución de problemas críticos.
- Planeación de sprints.

- Presentaciones y sustentaciones.

El equipo considera que esta distribución balanceada de responsabilidades permite reducir dependencias críticas entre integrantes y aumentar la resiliencia del proyecto frente a retrasos o dificultades técnicas.

9. Administración del Proyecto

9.1 Métodos y Herramientas de Estimación

Enfoque general de estimación

La estimación del proyecto se realizó mediante una combinación de métodos cuantitativos y cualitativos con el objetivo de obtener una planeación realista tanto del esfuerzo de desarrollo como de las actividades de documentación, validación y gestión.

Debido a que el proyecto involucra componentes de procesamiento de datos, simulación de tráfico y desarrollo de software académico, no resulta suficiente utilizar un único método de estimación tradicional. Por esta razón se adoptó una estrategia híbrida basada en:

- Estimación por analogía.
- Estimación por descomposición (WBS).
- Estimación basada en experiencia del equipo.
- Estimación incremental por sprint.
- Estimación de carga de documentación y validación.

El proceso de estimación fue realizado colectivamente por el equipo durante la planeación inicial del proyecto y refinado después de reuniones con el director.

Estimación del Software:

- **Estimación por analogía**

La primera aproximación de esfuerzo se realizó mediante comparación con proyectos similares encontrados en artículos académicos, repositorios públicos y tesis relacionadas con simulación de tráfico usando SUMO.

Se analizaron proyectos que incluían:

- Procesamiento ETL de datos de movilidad.
- Construcción de matrices O/D.
- Conversión de redes OpenStreetMap.
- Integración con SUMO.
- Validación mediante métricas de tráfico.

A partir de estas referencias se identificó que un proyecto de complejidad similar normalmente requiere entre 2 y 4 meses de trabajo parcial para equipos de 3 o 4 personas.

La comparación permitió establecer un rango inicial de esfuerzo aproximado y validar que el alcance propuesto era viable dentro del semestre académico.

- **Estimación basada en complejidad técnica**

Posteriormente, el equipo clasificó cada componente del sistema según su complejidad técnica.

Componente	Complejidad	Justificación
Construcción de matrices O/D	Alta	Transformaciones estadísticas y expansión muestral
Procesamiento de red vial	Alta	Conversión OSM-SUMO y validación geoespacial
Configuración de SUMO	Media-alta	Curva de aprendizaje del simulador
Validación GEH	Media	Implementación matemática y comparación estadística
Escenarios hipotéticos	Alta	Requiere calibración y análisis comparativo
Documentación IEEE	Media	Alto volumen documental y trazabilidad

Esta clasificación permitió priorizar tiempo adicional para las actividades de mayor riesgo técnico.

- **Estimación por descomposición (WBS)**

Después de la estimación inicial, el proyecto fue descompuesto en actividades específicas utilizando una Work Breakdown Structure (WBS).

Cada actividad fue estimada individualmente considerando:

- Complejidad técnica.
- Dependencias.
- Experiencia previa del equipo.
- Tiempo de aprendizaje requerido.
- Riesgos asociados.
- Actividades de pruebas y correcciones.

La suma de estas estimaciones produjo el total de horas-persona del proyecto.

Herramientas utilizadas para la estimación:

- **GitHub Projects:**

GitHub Projects se utilizó para estructurar el backlog inicial y dividir el trabajo en tareas concretas.

Cada actividad fue registrada como issue o tarea independiente, permitiendo estimar:

- Tiempo esperado.
- Dependencias.
- Prioridad.
- Responsables.
- Estado de avance.

La herramienta también facilita el seguimiento posterior del cumplimiento de las estimaciones.

Diagrama WBS

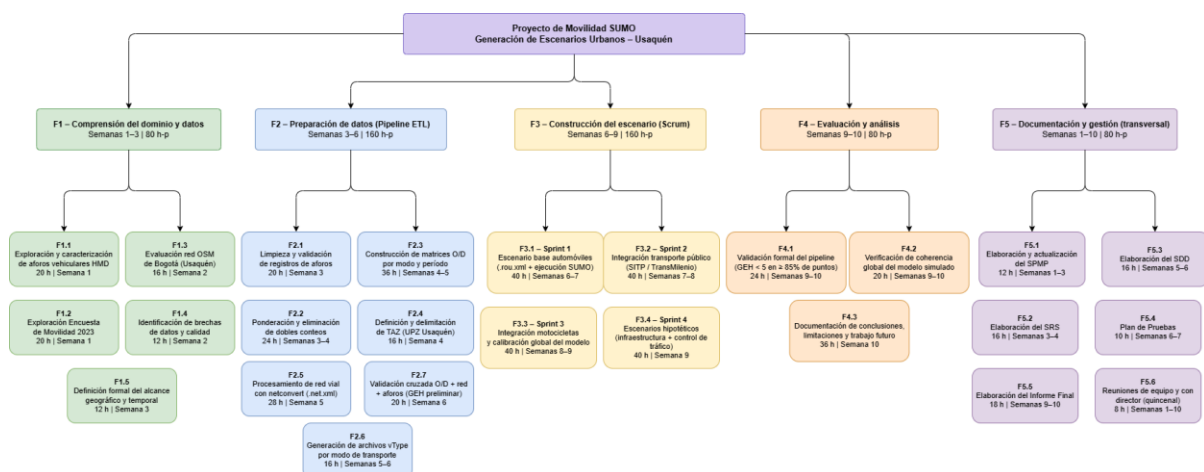


Figura 8. Diagrama WBS

La estructura WBS (Figura 8) fue utilizada para organizar el proyecto jerárquicamente y visualizar:

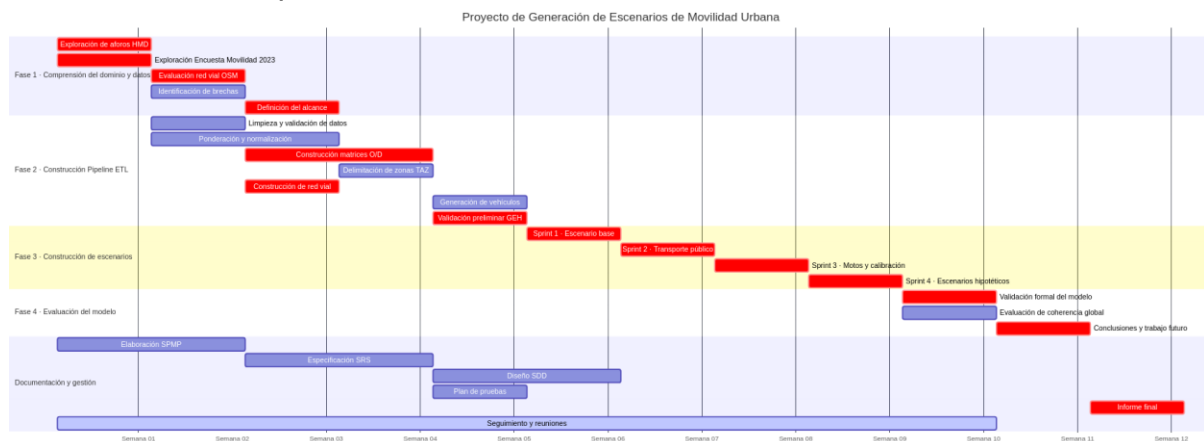
- Fases principales.
- Subtareas.
- Dependencias críticas.
- Relación entre entregables.

Gracias a esta estructura fue posible estimar el esfuerzo de manera más precisa y detectar actividades críticas.

Diagrama Gantt

El cronograma general fue representado mediante un diagrama Gantt para visualizar:

- Distribución temporal de actividades.
- Dependencias.
- Ruta crítica.
- Semanas de contingencia.
- Actividades paralelas.



El Gantt permitió validar que las actividades más complejas finalizarán antes del cierre formal del semestre. Este se encuentra al final de la sección 9.2.1.

Estimación del Proyecto:

- **Estimación de horas-persona:**

El esfuerzo total del proyecto fue estimado utilizando horas-persona como unidad principal.

La estimación se calculó considerando:

4 integrantes.

10 semanas efectivas de trabajo.

Aproximadamente 14 horas semanales por integrante.

La fórmula utilizada fue:

$560 \text{ horas-persona} = 4 \text{ integrantes} \times 10 \text{ semanas} \times 14 \text{ horas/semana}$.

Esta estimación contempla tanto actividades técnicas como documentación, reuniones, pruebas y correcciones.

Fase / Componente	H-P estimadas	Justificación
Fase 1 – Comprensión del dominio y datos	80 h-p	Exploración de fuentes de datos heterogéneas con análisis de calidad y brechas.
Fase 2 – Preparación de datos (ETL)	160 h-p	Componente más complejo: limpieza, matrices O/D, red vial, validación cruzada.
Fase 3 – Construcción del escenario (5 sprints)	240 h-p	5 sprints $\times$ ~40 h/sprint; incluye calibración, escenarios hipotéticos y desarrollo del módulo web (backend FastAPI + frontend de visualización).
Fase 4 – Evaluación y análisis	80 h-p	Validación formal, análisis de resultados, documentación de conclusiones.
Documentación y gestión (transversal)	80 h-p	SPMP, SRS, SDD, Plan de pruebas, Informe final, reuniones.
Total	600 h-p	4 personas $\times$ 10 semanas $\times$ 14 h/semana + 40 h adicionales Sprint 5 (módulo web).

- Estimación de riesgo y contingencia

El equipo decidió planificar únicamente 10 semanas efectivas de trabajo dentro de un semestre de 16 semanas.

Esta decisión se tomó deliberadamente para disponer de tiempo adicional destinado a:

- Corrección de errores.
- Retrasos técnicos.
- Ajustes de calibración.
- Revisión documental.
- Cambios solicitados por el director.
- Problemas de integración.
- Riesgos académicos.

La estrategia busca evitar que el proyecto llegue al cierre del semestre sin margen de maniobra.

- Validación de las Estimaciones

Las estimaciones fueron revisadas colectivamente y ajustadas durante reuniones con el director del proyecto.

Los principales criterios utilizados para validar las estimaciones fueron:

- Viabilidad dentro del tiempo disponible.
- Consistencia entre alcance y recursos.
- Balance de carga entre integrantes.
- Riesgo técnico aceptable.
- Compatibilidad con otros compromisos académicos.

Las estimaciones podrán ajustarse durante el desarrollo si aparecen riesgos o cambios importantes en el alcance.

9.2 Inicio del Proyecto

9.2.1 Entrenamiento del Personal

Conocimiento / Habilidad	Estado actual	Plan de entrenamiento	Semana
SUMO (configuración y uso básico)	Nivel básico	Tutoriales oficiales SUMO + documentación TraCI + práctica con dataset de ejemplo.	1–2
netconvert y duarouter	Sin experiencia	Práctica guiada con la red OSM de Usaquén descargada.	2
geopandas y análisis geoespacial	Nivel básico-intermedio	Tutoriales de DataCamp + práctica con los datos de aforos y UPZ.	1–3
pytest y GitHub Actions	Nivel básico	Configuración del repositorio con el primer pipeline de CI en la semana 1.	1

9.2.2 Infraestructura

Objetivo de la infraestructura del proyecto:

La infraestructura del proyecto tiene como propósito proporcionar un entorno de desarrollo reproducible, colaborativo y estable que permita ejecutar correctamente el pipeline ETL, las simulaciones SUMO y las actividades de documentación y validación.

Debido a que el proyecto involucra procesamiento de datos, simulación microscópica de tráfico y trabajo colaborativo entre varios integrantes, la infraestructura debe garantizar:

- Compatibilidad entre herramientas.
- Versionamiento adecuado.
- Reproducibilidad de resultados.
- Integración continua.
- Disponibilidad de datos.
- Colaboración eficiente.

Todas las herramientas seleccionadas son de código abierto o gratuitas para uso académico.

Actividades:

Tarea	Responsable	Semana
Crear repositorio GitHub con estructura base (src/, tests/, docs/, data/)	Erick Salazar	1
Configurar GitHub Actions con pytest + pylint	Erick Salazar	1
Instalar SUMO 1.19+ en los equipos del equipo	Todos	1
Configurar entorno virtual Python con requirements.txt	Erick Salazar	1
Descargar y versionar datasets base (aforos HMD, encuesta, OSM Usaqué)	Felipe Garrido	1–2
Configurar GitHub Projects para backlog y burndown chart	Scrum Master semana 1	1

- **Configuración inicial del repositorio GitHub:**

La primera actividad de infraestructura consiste en la creación del repositorio principal del proyecto en GitHub.

Tareas asociadas

- Crear repositorio privado inicial.
- Definir estructura de carpetas.
- Configurar ramas principales.
- Definir reglas de merge.
- Crear README inicial.
- Configurar permisos de colaboradores.
- Definir convenciones de commits.

Responsables:

- Responsable de infraestructura.
- Scrum Master del Sprint 1.

Momento de ejecución:

Semana 1.

Frecuencia:

Actividad inicial con mantenimiento continuo durante todo el proyecto.

- **Configuración del entorno Python**

El proyecto utiliza Python como lenguaje principal del pipeline ETL y de integración con SUMO.

Tareas asociadas:

- Instalar Python 3.11+.
- Crear entorno virtual.
- Configurar requirements.txt.
- Instalar librerías necesarias.
- Verificar compatibilidad entre dependencias.
- Actualizar dependencias cuando sea necesario.

Responsables:

- Responsable de infraestructura.
- Todos los integrantes.

Momento de ejecución:

Semana 1.

Frecuencia:

Revisión y mantenimiento semanal.

- **Instalación y configuración de SUMO**

SUMO constituye la herramienta central de simulación del proyecto.

Tareas asociadas:

- Descargar versión estable de SUMO.
- Configurar variables de entorno.
- Verificar funcionamiento de netconvert.
- Verificar funcionamiento de duarouter.
- Configurar TraCI.
- Ejecutar simulaciones de prueba.
- Verificar compatibilidad con Python.

Responsables:

- Responsable de simulación.
- Responsable de infraestructura.

Momento de ejecución:

Semana 1.

Frecuencia:

Validación continua al inicio de cada sprint.

- **Configuración de GitHub Actions:**

La integración continua permite ejecutar pruebas automáticas y mantener calidad constante del código.

Tareas asociadas:

- Configurar workflows de CI.
- Integrar pytest.
- Integrar pylint.
- Verificar ejecución automática.
- Configurar validación por pull requests.
- Corregir errores de pipeline.

Responsables:

- Responsable de infraestructura.
- Responsable de pruebas.

Momento de ejecución:

Semana 1.

Frecuencia:

Monitoreo continuo por commit.

- Descarga y gestión de datasets

Los datasets constituyen uno de los activos más importantes del proyecto.

Tareas asociadas:

- Descargar Encuesta de Movilidad.
- Descargar aforos HMD.
- Descargar red OpenStreetMap.
- Verificar integridad de archivos.
- Versionar datasets procesados.
- Mantener estructura organizada de datos.
- Generar respaldos locales.

Responsables:

- Responsable de datos.
- Responsable de infraestructura.

Momento de ejecución:

Semanas 1 y 2.

Frecuencia:

Actualización cuando existan nuevas versiones o correcciones.

- Configuración del sistema de gestión de proyecto

El seguimiento de tareas se realizará mediante GitHub Projects.

Tareas asociadas:

- Crear tablero Kanban.

- Definir columnas del flujo.
- Registrar backlog inicial.
- Configurar etiquetas.
- Configurar milestones.
- Registrar issues por sprint.

Responsables:

- Scrum Master.
- Responsable de infraestructura.

Momento de ejecución:

Semana 1.

Frecuencia:

Actualización diaria.

Mantenimiento de la infraestructura:

La infraestructura requiere mantenimiento continuo para garantizar estabilidad y reproducibilidad.

Actividades de mantenimiento

- Actualización de dependencias.
- Respaldo de información crítica.
- Monitoreo de pipelines CI.
- Limpieza de ramas obsoletas.
- Verificación de compatibilidad.
- Corrección de conflictos de versiones.
- Validación periódica de entornos.

Responsables:

- Responsable de Infraestructura.
- Todos los integrantes para validación local.

Frecuencia:

Semanal o cuando se detecten problemas.

Riesgos:

Riesgo	Impacto	Mitigación
Incompatibilidad de versiones	Alto	Uso de requirements.txt y documentación detallada
Fallos en GitHub Actions	Medio	Validación manual temporal
Pérdida de datos	Alto	Backups periódicos y versionamiento
Problemas de instalación de SUMO	Alto	Guías compartidas y soporte interno
Dependencias rotas	Medio	Congelamiento de versiones estables

9.3 Planes de Trabajo del Proyecto

9.3.1 Calendarización con ruta crítica

La siguiente tabla presenta la descomposición completa de actividades (WBS) con la duración estimada, las semanas de ejecución, los prerequisites (actividades que deben estar completas antes de iniciar) y los co-requisitos (actividades que deben ejecutarse en paralelo). Las actividades resaltadas en gris corresponden a fases de nivel superior.

El camino crítico del proyecto es: F1.1 → F1.2 → F1.3 → F1.5 → F2.5 → F2.7 → F3.1 → F3.2 → F3.3 → F3.4 → F4.1 → F4.3. Cualquier retraso en estas actividades impacta directamente la fecha de finalización del proyecto.

ID	Actividad	H-P	Semana(s)	Prerequisito(s)	Co-req.
F1	FASE 1 – Comprensión del dominio y los datos	80 h	1–3	—	—
F1.1	Exploración y caracterización de aforos vehiculares HMD	20 h	1	—	F1.2
F1.2	Exploración de la Encuesta de Movilidad 2023	20 h	1	—	F1.1
F1.3	Evaluación de la red OSM de Bogotá para Usaqué	16 h	2	F1.1, F1.2	—
F1.4	Identificación de brechas de datos y calidad	12 h	2	F1.1, F1.2	F1.3
F1.5	Definición formal del alcance geográfico y temporal	12 h	3	F1.3, F1.4	—

F2	FASE 2 – Preparación de datos (Pipeline ETL)	160 h	3–6	F1	—
F2.1	Limpieza y validación de registros de aforos	20 h	3	F1.1	F2.2
F2.2	Aplicación de factores de ponderación y eliminación de dobles conteos	24 h	3–4	F1.2	F2.1
F2.3	Construcción de matrices O/D por modo y período	36 h	4–5	F2.1, F2.2	F2.4
F2.4	Definición y delimitación de TAZ (UPZ de Usaquén)	16 h	4	F1.5	F2.3
F2.5	Procesamiento de red vial con netconvert (.net.xml)	28 h	5	F1.3	—
F2.6	Generación de archivos vType por modo de transporte	16 h	5–6	F2.3	F2.5
F2.7	Validación cruzada O/D + red + aforos (GEH preliminar)	20 h	6	F2.3, F2.5, F2.6	—
F3	FASE 3 – Construcción del escenario (Scrum, 5 sprints)	200 h	6–10	F2	—
F3.1	Sprint 1: Escenario base automóviles (.rou.xml + ejecución SUMO)	40 h	6–7	F2.7	—
F3.2	Sprint 2: Integración transporte público (SITP / TransMilenio)	40 h	7–8	F3.1	—
F3.3	Sprint 3: Integración motocicletas y calibración global del modelo	40 h	8–9	F3.2	—
F3.4	Sprint 4: Escenarios hipotéticos (infraestructura + control de tráfico)	40 h	9	F3.3	—
F3.5	Sprint 5: Desarrollo del módulo web (backend API + frontend de visualización + parametrización)	40 h	9–10	F3.4	—
F4	FASE 4 – Evaluación y análisis	80 h	9–10	F3	—
F4.1	Validación formal del pipeline (GEH < 5 en ≥ 85% de puntos)	24 h	9–10	F3.4	F4.2
F4.2	Verificación de coherencia global del modelo simulado	20 h	9–10	F3.4	F4.1

F4.3	Documentación de conclusiones, limitaciones y trabajo futuro	36 h	10	F4.1, F4.2	—
F5	DOCUMENTACIÓN Y GESTIÓN (transversal)	80 h	1–10	—	—
F5.1	Elaboración y actualización del SPMP	12 h	1–3	—	F5.2
F5.2	Elaboración del SRS	16 h	3–4	F5.1	—
F5.3	Elaboración del SDD	16 h	5–6	F5.2	—
F5.4	Elaboración del Plan de Pruebas	10 h	6–7	F5.2	—
F5.5	Elaboración del Informe Final	18 h	9–10	F4.3	—
F5.6	Reuniones de equipo (lunes) y con director (quincenal)	8 h	1–10	—	Todos

9.3.2 Asignación de Recursos

Fase / Actividad	Samuel Campos	Erick Salazar	Santiago Chitiva	Felipe Garrido
Fase 1 – Dominio y datos	Líder	Co-líder	Apoyo	Apoyo
Fase 2 – Pipeline ETL	Co-líder (O/D)	Co-líder (limpieza)	Líder (red OSM)	Apoyo (validación)
Fase 3 – Escenarios (Scrum)	Apoyo	Apoyo (pruebas)	Co-líder (simulación)	Líder (calibración)
Fase 3 - Módulo Web	Apoyo	Líder	Apoyo	Apoyo
Fase 4 – Evaluación	Apoyo	Líder (métricas)	Apoyo	Co-líder (análisis)
Documentación	SRS	Plan de pruebas	SDD / Informe	SPMP / Informe

9.3.4 Asignación de presupuesto y justificación

10. Monitoreo y Control del Proyecto

10.1 Administración de Requerimientos

Los requerimientos del sistema se gestionan a través del SRS (entregable de la semana 4). El proceso de administración de requerimientos incluye: identificación inicial, revisión con el director, aprobación, y control de cambios durante el desarrollo.

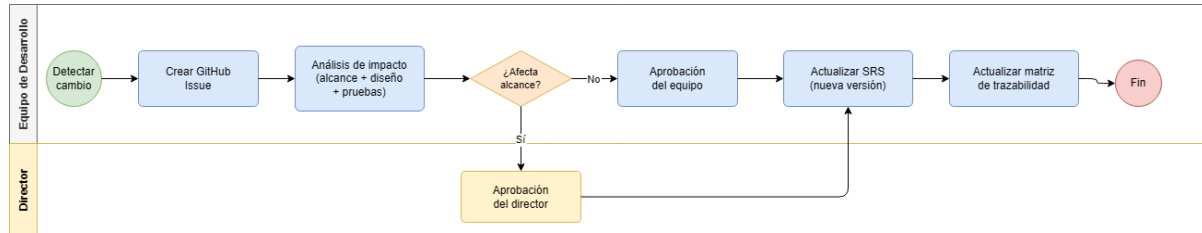


Figura 2. Diagrama BPMN – Proceso de administración de requerimientos

Figura 3. Diagrama BPMN – Proceso de administración de requerimientos.

[DIAGRAMA BPMN – Figura 3 – Administración de requerimientos] El flujo muestra: detección del cambio → creación de GitHub Issue → análisis de impacto (alcance + diseño + pruebas) → aprobación (director si afecta alcance; equipo si es menor) → actualización del SRS con nueva versión → actualización de la matriz de trazabilidad. Roles: equipo de desarrollo, director.

Artefactos generados: SRS (requerimientos funcionales y no funcionales), matriz de trazabilidad (SRS ↔ diseño ↔ pruebas), registro de cambios de requerimientos en GitHub Issues.

Proceso de control de cambios: toda solicitud de cambio a un requerimiento aprobado debe documentarse como un GitHub Issue, ser analizada por el equipo (impacto en alcance, calendario y diseño), aprobada por el director si afecta el alcance, e incorporada al SRS con actualización de la versión. Los cambios se discuten al inicio de cada sprint.

10.2 Monitoreo y Control de Progreso

El monitoreo y control del proyecto tiene como propósito verificar continuamente que el desarrollo avance de acuerdo con el alcance, cronograma y objetivos definidos en el SPMP.

El proceso de seguimiento busca identificar tempranamente desviaciones, retrasos o problemas técnicos que puedan afectar la entrega del proyecto, permitiendo aplicar acciones correctivas oportunamente.

El modelo de seguimiento combina prácticas de Scrum, métricas cuantitativas y validaciones técnicas específicas para proyectos de simulación de tráfico.

Métricas de Progreso:

Las métricas seleccionadas permiten medir tanto el avance del software como la calidad de los resultados producidos.

Métrica	Unidad	Frecuencia	Responsable
Tareas completadas vs. planificadas	Issues cerrados / total sprint	Fin de cada sprint	Scrum Master
Burndown del sprint	Horas restantes	Diario	Scrum Master
Cobertura de pruebas unitarias	% de líneas cubiertas	Por commit (CI automático)	Responsable de pruebas
Porcentaje de validación GEH	% de puntos con GEH < 5	Fin de cada sprint de simulación	Responsable de simulación y pruebas
Velocidad del equipo	Puntos de historia por sprint	Fin de cada sprint	Scrum Master
Número de requerimientos completados	Requerimientos completados / requerimientos totales	Semanal	Responsable de documentación y Scrum Master
Errores detectados y corregidos	Número de issues de error cerrados	Semanal	Responsable de pruebas

- **Tareas completadas vs. tareas planificadas**

Descripción

Mide la cantidad de tareas finalizadas respecto a las tareas comprometidas para un sprint o fase.

Unidad

Número de issues cerrados / total de issues planificados.

Objetivo

Verificar cumplimiento del backlog comprometido.

Frecuencia

Al cierre de cada sprint.

Responsable

Scrum Master.

- **Horas restantes del sprint (Burndown)**

Descripción

Mide el trabajo pendiente dentro de cada sprint.

Unidad

Horas restantes estimadas.

Objetivo

Detectar retrasos tempranos y sobrecarga de trabajo.

Frecuencia

Seguimiento diario.

Responsable

Scrum Master.

- **Cobertura de pruebas unitarias**

Descripción

Mide el porcentaje de código cubierto por pruebas automatizadas.

Unidad

Porcentaje de líneas cubiertas.

Objetivo

Garantizar calidad y estabilidad del pipeline.

Frecuencia

Automática por commit mediante CI.

Responsable

Responsable de pruebas.

- **Validación GEH**

Descripción

Mide el porcentaje de puntos de aforo cuya simulación alcanza valores GEH menores a 5.

Unidad

Porcentaje de puntos válidos.

Objetivo

Validar calibración del modelo de tráfico.

Frecuencia

Al cierre de cada sprint de simulación.

Responsable

Responsable de simulación y responsable de pruebas.

- **Velocidad del equipo**

Descripción

Mide la cantidad de trabajo completado por sprint.

Unidad

Puntos de historia completados.

Objetivo

Estimar capacidad real del equipo.

Frecuencia

Fin de cada sprint.

Responsable

Scrum Master.

- **Número de requerimientos implementados**

Descripción

Mide la cantidad de requerimientos funcionales implementados y verificados.

Unidad

Requerimientos completados / requerimientos totales.

Objetivo

Verificar avance funcional del sistema.

Frecuencia

Semanal.

Responsable

Responsable de Documentación y Scrum Master.

- **Errores detectados y corregidos**

Descripción

Mide la cantidad de defectos identificados y solucionados.

Unidad

Número de issues de error cerrados.

Objetivo

Monitorear estabilidad y calidad.

Frecuencia

Semanal.

Responsable

Responsable de Pruebas.

Actividades de reporte de progreso:

- **Reuniones semanales de seguimiento:**

El equipo realiza reuniones semanales para revisar el estado del proyecto.

Actividades realizadas:

- Revisión de tareas completadas.
- Identificación de bloqueos.
- Revisión de métricas.
- Actualización del backlog.
- Ajuste de prioridades.
- Definición de compromisos.

Responsables:

Todos los integrantes.

Frecuencia:

Dos veces por semana.

- **Reuniones con el director**

Al cierre de cada sprint se realiza una reunión con el director del proyecto.

Actividades realizadas

- Presentación de avances.
- Revisión de entregables.
- Discusión de riesgos.
- Retroalimentación técnica.
- Validación de decisiones importantes.

Responsables

Equipo completo y director.

Frecuencia

Semanal según disponibilidad.

- **Actualización de GitHub Projects**

GitHub Projects actúa como mecanismo central de seguimiento.

Actividades realizadas

- Actualizar estado de tareas.
- Registrar issues.
- Registrar bloqueos.
- Actualizar responsables.
- Registrar progreso del sprint.

Responsables

Scrum Master.

Frecuencia

Diaria.

- **Verificación de implementación exitosa**

Para considerar que un requerimiento fue implementado exitosamente deben cumplirse los siguientes criterios:

- El código compila y ejecuta correctamente.
- Las pruebas unitarias asociadas pasan exitosamente.
- El cambio fue revisado mediante pull request.
- El resultado es consistente con el SRS.
- La funcionalidad fue validada por al menos otro integrante.
- La documentación correspondiente fue actualizada.

En el caso de componentes de simulación adicionalmente se requiere:

- Ejecución correcta en SUMO.
- Generación válida de archivos XML.
- Métricas GEH dentro del rango esperado.

Acciones correctivas:

Cuando las métricas indiquen desviaciones importantes respecto a la planeación inicial, el equipo aplicará acciones correctivas.

- **Retrasos en el Sprint**

Condición

Avance inferior al 70% del trabajo comprometido.

Acción correctiva

- Reunión extraordinaria.
- Repriorización del backlog.
- Redistribución de tareas.
- Reducción de alcance no crítico.

Responsables

Scrum Master y equipo.

- **Problemas técnicos críticos**

Condición

Errores que impidan continuar el desarrollo.

Acción correctiva

- Investigación colaborativa.

- Escalamiento al director.
- Implementación temporal alternativa.
- Ajuste del cronograma.

Responsables

Responsable técnico del área afectada.

- **Baja calidad del modelo**

Condición

GEH > 5 en más del 15% de puntos.

Acción correctiva

- Revisión de matrices O/D.
- Ajuste de parámetros de simulación.
- Validación de aforos.
- Recalibración del escenario.

Responsables

Responsable de simulación y responsable de datos.

- **Problemas de Calidad del Código**

Condición

Cobertura insuficiente o errores frecuentes.

Acción correctiva

- Incrementar pruebas.
- Revisión obligatoria de código.
- Refactorización.
- Congelamiento temporal de nuevas funcionalidades.

Responsables

Responsable de pruebas.

- **Desviaciones documentales**

Condición

Inconsistencias entre entregables.

Acción correctiva

- Revisión cruzada de documentos.
- Actualización de trazabilidad.
- Corrección de versiones.
- Validación colectiva.

Responsables

Responsable de documentación.

Cierre del monitoreo

Al finalizar cada sprint se consolidará un resumen de:

- Tareas completadas.
- Problemas encontrados.
- Riesgos identificados.
- Acciones correctivas aplicadas.
- Estado de métricas.
- Decisiones tomadas.

Estos resultados servirán como insumo para retrospectivas y para ajustar la planeación de los siguientes sprints.

10.3 Cierre del Proyecto

Al finalizar cada entrega formal del proyecto, el equipo realiza: retrospectiva de la fase (con registro de lecciones aprendidas), actualización del SPMP y del registro de riesgos, y verificación de que todos los artefactos están versionados en GitHub.

Al cierre final (semana 10-16) se entregan todos los artefactos al director y a la coordinación académica, se realiza la exposición ante los jurados y se archiva el repositorio con el código fuente y la documentación.

11. Entrega del Producto

La entrega del producto se realizará en la semana 10-16 del proyecto e incluirá los siguientes artefactos:

- Repositorio GitHub con el código fuente completo del pipeline ETL, documentado con docstrings y README de instalación, y el código fuente del módulo web.
- Conjunto de archivos de configuración SUMO (.net.xml, .taz.xml, .rou.xml, .vtype.xml) para la localidad de Usaquén.
- Reporte de validación del modelo con métricas GEH y RMSE por punto de aforo.
- Resultados de los escenarios hipotéticos analizados (Sprint 3 y Sprint 4).
- Informe final del proyecto (IEEE 1063-2001).
- Todos los entregables formales del semestre (SPMP, SRS, SDD, Plan de Pruebas).

La aceptación de cada entregable sigue los criterios descritos en la sección 8.3. La aceptación final está condicionada a la aprobación del informe por parte de los jurados evaluadores en la exposición de la semana 16.

12. Procesos de Soporte

12.1 Ambiente de Trabajo

El equipo opera bajo las siguientes reglas de trabajo:

- Dos reuniones semanales fijas: lunes (planeación y seguimiento, 1 hora) y viernes (revisión y retrospectiva, 30 minutos).
- Toda comunicación técnica y de decisiones relevantes debe quedar documentada en GitHub (Issues, Pull Requests, comentarios).
- El código se escribe en ramas feature/* y solo se integra a develop cuando es aprobado por al menos un compañero.
- Los documentos formales se redactan colaborativamente antes de exportarse al formato final.
- Los integrantes deben avisar con 48 horas de anticipación si no pueden asistir a una reunión o cumplir un compromiso.
- Las decisiones técnicas que afecten el diseño global se documentan como Architecture Decision Records (ADR) en el repositorio.
- Siempre se priorizarán soluciones bien fundamentadas sobre soluciones rápidas.
- Siempre se priorizará la colaboración sobre el trabajo individual.
- Siempre se priorizará el impacto a largo plazo sobre los resultados a corto plazo.
- Cada integrante es responsable de sus tareas, pero puede pedir ayuda cuando lo necesite.
- Los compromisos adquiridos en reuniones deben cumplirse en los tiempos acordados.
- Se documentarán los acuerdos y decisiones importantes al finalizar cada reunión por escrito.
- Las decisiones se tomarán entre todos buscando consenso. Si no se llega a consenso, se decidirá por votación.
- Quien tenga mayor conocimiento técnico sobre un tema liderará las decisiones de esa área.
- Si una decisión resulta incorrecta, el equipo buscará una solución sin señalar ni culpar a ningún integrante.
- El feedback entre integrantes del grupo debe ser claro, respetuoso y basado en hechos concretos, nunca en ataques personales.
- Si algo incomoda o genera conflicto, debe comunicarse pronto, antes de que escale.
- Ninguna idea será descartada sin análisis previo.
- En las reuniones, todos deben tener espacio para hablar. Si alguien no participa, se le invitará respetuosamente a opinar.
- Las reuniones deben terminar con acciones concretas y un responsable asignado para cada una.
- Los errores se usan para mejorar, no para señalar a un compañero.

- Se respetarán los tiempos y espacios personales de cada integrante sin descuidar el compromiso grupal.
- Trabajar de forma aislada sin compartir avances con el equipo no está permitido.

12.2 Análisis y Administración de Riesgos

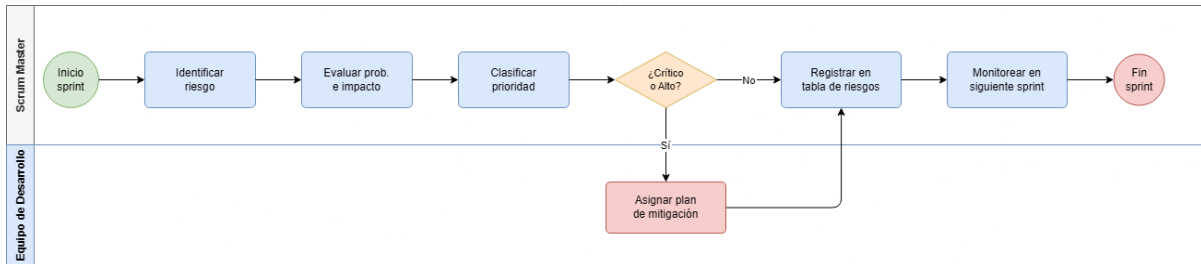


Figura 3. Diagrama BPMN – Proceso de identificación y seguimiento de riesgos

Figura 4. Diagrama BPMN – Proceso de identificación y seguimiento de riesgos.

[DIAGRAMA BPMN – Figura 4 – Administración de riesgos] El flujo muestra el ciclo por sprint: identificar riesgo → evaluar probabilidad e impacto → clasificar prioridad → asignar plan de mitigación → monitorear en el siguiente sprint → actualizar el registro. Roles: Scrum Master (responsable del registro), equipo de desarrollo.

Registro de Riesgos

ID	Descripción del riesgo	Prob.	Impacto	Prioridad
R1	Calidad insuficiente de los datos de la Encuesta de Movilidad para construir matrices O/D representativas.	Alta	Alto	Crítica
R2	Red vial de OSM incompleta o con errores geométricos en Usaquéen.	Media	Alto	Alta
R3	SUMO no puede simular el escenario generado por el pipeline (archivos inválidos o incompatibles).	Media	Alto	Alta
R4	Estimación de tiempo insuficiente para la calibración del modelo (GEH no alcanzable en 10 semanas).	Media	Medio	Media
R5	Baja disponibilidad de un integrante del equipo por compromisos académicos o personales.	Baja	Alto	Media
R6	Datos de aforos HMD insuficientes para cubrir todos los puntos de validación de Usaquéen.	Media	Medio	Media
R7	Conflictos de integración en el repositorio por trabajo simultáneo en ramas.	Alta	Bajo	Media

Plan de Mitigación

ID	Prevención	Mitigación (si ocurre)	Responsable
R1	Exploración temprana de la encuesta en Fase 1 con análisis de calidad y cobertura.	Reducir el alcance a las UPZ con mayor cobertura muestral.	Samuel / Erick
R2	Inspección manual de la red OSM en QGIS antes de iniciar Fase 2.	Corrección de geometrías con JOSM o parámetros de netconvert.	Santiago
R3	Pruebas de integración del pipeline contra SUMO desde el Sprint 1.	Depuración de archivos XML con herramientas de validación de SUMO.	Felipe / Santiago
R4	Definir GEH como criterio aspiracional (no bloqueante) en sprints intermedios.	Ajustar el criterio de 'done' o reducir modos integrados en acuerdo con el director.	Director + equipo
R5	Documentación continua del conocimiento en el repositorio (README, docstrings).	Redistribuir tareas entre los tres integrantes restantes.	Scrum Master

12.3 Administración de Configuración y Documentación

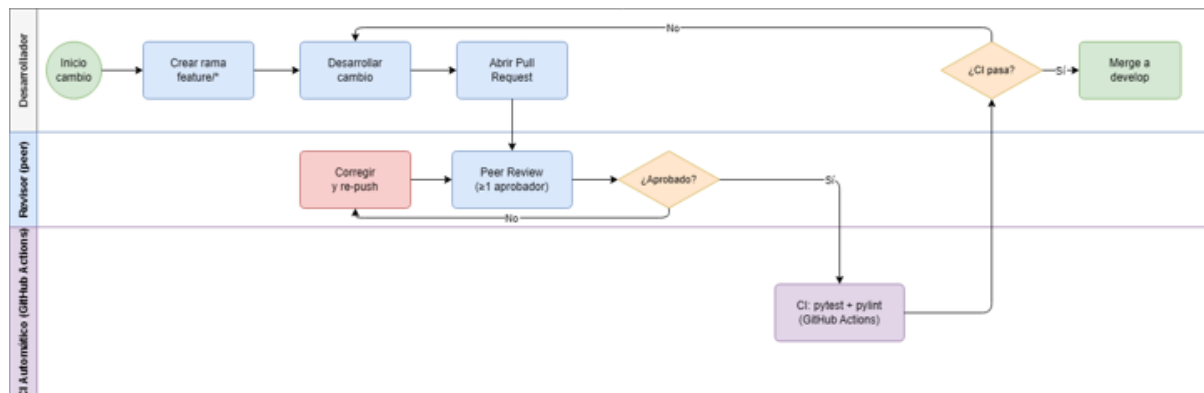


Figura 5. Diagrama BPMN – Proceso de control de configuración.

[DIAGRAMA BPMN – Figura 5 – Control de configuración] El flujo muestra: cambio en código o documento → creación de rama feature/* → desarrollo → pull request → peer review (al menos 1 aprobador) → ejecución automática de CI (GitHub Actions) → merge a develop → validación final → release a main. Roles: desarrollador, revisor, CI automático.

Ítems de Configuración

Ítem de configuración	Tipo	Ubicación en el repositorio
-----------------------	------	-----------------------------

Código fuente del pipeline ETL	Software	src/pipeline/
Scripts de simulación SUMO	Software	src/simulation/
Pruebas unitarias (pytest)	Software	tests/
SPMP	Documento	docs/spmp/
SRS	Documento	docs/srs/
SDD	Documento	docs/sdd/
Plan de Pruebas	Documento	docs/testing/
Informe Final	Documento	docs/informe_final/
Datos de entrada (aforos, encuesta, OSM)	Datos	data/raw/ (Google Drive; no versionados en Git)
Archivos procesados SUMO (.net.xml, .od, .rou.xml)	Artefacto generado	data/processed/
requirements.txt	Configuración	raíz del repositorio
Configuración de GitHub Actions (.yml)	Configuración CI	.github/workflows/

Ciclo de Vida de Artefactos

Artefacto	Se crea en	Se actualiza en
SPMP	Semana 1–3	Cada entrega formal del proyecto
SRS	Semana 3–4	Si hay cambios de requerimientos aprobados
SDD	Semana 5–6	Con cambios de diseño durante Fase 3
Plan de Pruebas	Semana 6–7	Con cambios de diseño o SRS
Código fuente	Desde semana 1	Continuo (por commit)
Informe Final	Semana 9	Semana 9–10
Registro de riesgos	Semana 1	Inicio de cada sprint

12.4 Métricas y Proceso de Medición

Métrica	Sección de uso	Quién recolecta	Cuándo	Herramienta
Tareas completadas / sprint	10.2	Scrum Master	Fin de sprint	GitHub Projects
Burndown del sprint	10.2	Scrum Master	Diario	GitHub Projects
Cobertura de pruebas (%)	10.2	CI automático	Por commit	pytest-cov + GitHub Actions
GEH por punto de aforo	8.3 / 10.2	Responsable de simulación	Fin de sprint de simulación	Script Python + salida SUMO

RMSE del modelo	8.3 / 10.2	Responsable de simulación	Fin de Fase 3 y Fase 4	Script Python
Velocidad del equipo	10.2	Scrum Master	Fin de sprint	GitHub Projects

12.5 Control de Calidad

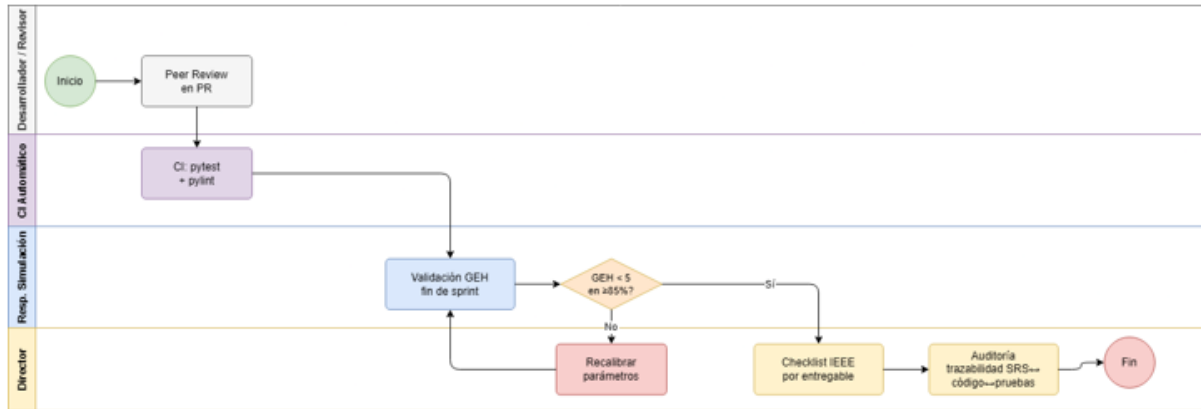


Figura 6. Diagrama BPMN – Proceso de control de calidad.

[DIAGRAMA BPMN – Figura 6 – Control de calidad] Muestra los tres niveles: (1) calidad del código: peer review en PR + CI automático (pytest + pylint); (2) calidad del modelo de simulación: validación GEH al cierre de cada sprint de simulación; (3) calidad de documentos formales: revisión con checklist por estándar antes de cada entrega + auditoría de trazabilidad SRS ↔ código ↔ pruebas. Roles: desarrollador, revisor, director, CI automático.

Proceso de calidad	Momento	Responsable	Herramienta
Revisión de cambios (peer review)	Por cada PR	Al menos 1 compañero	GitHub PR reviews
Pruebas unitarias automatizadas (CI)	Por cada commit	GitHub Actions	pytest + pylint
Validación GEH del modelo	Fin de cada sprint de simulación	Responsable de simulación	Script Python + aforos reales
Revisión formal de documentos	Antes de cada entrega formal	Equipo + director	Checklist por estándar IEEE
Retrospectiva de sprint	Fin de cada sprint	Scrum Master	Registro de lecciones aprendidas
Auditoría de trazabilidad SRS ↔ código ↔ pruebas	Antes de cada entrega formal	Responsable de documentación	Matriz de trazabilidad

13. Anexos

Anexo A – Estructura del Repositorio

Carpeta / Archivo	Contenido
src/pipeline/	Módulos Python del pipeline ETL (extracción, transformación, carga)
src/simulation/	Scripts de configuración y ejecución de SUMO
tests/	Pruebas unitarias con pytest
data/raw/	Datos de entrada originales (enlace a Google Drive; no versionados en Git)
data/processed/	Archivos SUMO generados (.net.xml, .od, .rou.xml, .vtype.xml)
docs/	Documentos formales del proyecto (SPMP, SRS, SDD, Plan de pruebas, Informe)
notebooks/	Jupyter Notebooks de exploración de datos (Fase 1)
requirements.txt	Dependencias Python del proyecto con versiones fijas
.github/workflows/	Configuración de GitHub Actions (CI/CD)
README.md	Guía de instalación y uso del sistema

Anexo B – Registro de Riesgos Actualizable

El registro de riesgos se mantiene actualizado en la sección 12.2 de este documento y como archivo en docs/riesgos/ del repositorio. Se revisa al inicio de cada sprint.

14. Referencias

- [1] IEEE, "IEEE Std 1058-1998 – IEEE Standard for Software Project Management Plans", 1998.
- [2] ISO/IEC, "ISO/IEC 12207:2008 – Systems and software engineering – Software life cycle processes", 2008.
- [3] OMG, "Business Process Model and Notation (BPMN) Version 2.0", 2011.
- [4] Lopez, P. A. et al., "Microscopic Traffic Simulation using SUMO", ITSC 2018.
- [5] Ortúzar, J. de D., & Willumsen, L. G., "Modelling Transport" (4th ed.), Wiley, 2011.
- [6] Secretaría Distrital de Movilidad de Bogotá, "Encuesta de Movilidad de Bogotá 2023", Alcaldía Mayor de Bogotá.
- [7] Wirth, R., & Hipp, J., "CRISP-DM: Towards a standard process model for data mining", 2000.
- [8] Schwaber, K., & Sutherland, J., "The Scrum Guide", 2020.
- [9] IEEE, "IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications", 1998.
- [10] IEEE, "IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions", 2009.
- [11] ISO/IEC/IEEE, "ISO/IEC/IEEE 29119-1:2022 – Software and systems engineering – Software testing", 2022.